

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import matplotlib.pyplot as plt
import networkx as nx

# Manually entering a small dataset
data = {
    "age": [29, 45, 34, 60, 50, 41, 52, 39, 48, 59],
    "cholesterol": [200, 240, 210, 280, 260, 230, 300, 220, 250, 270],
    "bp": [120, 140, 130, 150, 140, 135, 160, 125, 145, 155],
    "heart_disease": [0, 1, 0, 1, 1, 0, 1, 0, 1, 1]
}

# Create a DataFrame
df = pd.DataFrame(data)

# Features and target
X = df[["age", "cholesterol", "bp"]]
y = df["heart_disease"]

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Naive Bayes Model
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)

# Predictions
y_pred = nb_model.predict(X_test)

# Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

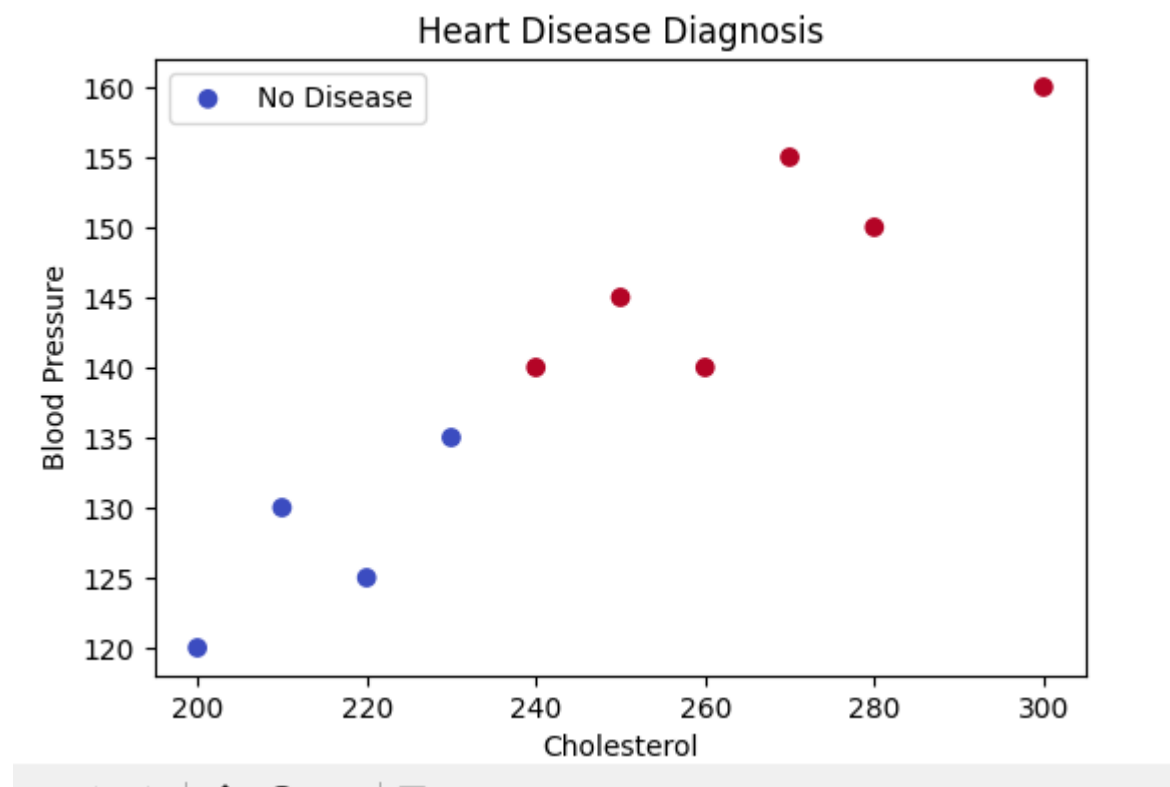
# Visualization of Results
plt.figure(figsize=(6, 4))
plt.scatter(df["cholesterol"], df["bp"], c=df["heart_disease"], cmap="coolwarm")
plt.xlabel("Cholesterol")
plt.ylabel("Blood Pressure")
plt.title("Heart Disease Diagnosis")
plt.legend(["No Disease", "Disease"], loc="best")

```

```
plt.show()

# Bayesian Network Visualization
G = nx.DiGraph()
G.add_edges_from([
    ("age", "heart_disease"),
    ("cholesterol", "heart_disease"),
    ("bp", "heart_disease")
])

plt.figure(figsize=(8, 6))
nx.draw(
    G, with_labels=True,
    node_size=3000, node_color="lightblue",
    font_size=10, font_weight="bold"
)
plt.title("Bayesian Network for Heart Disease Diagnosis")
plt.show()
```



```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
```

```

class SimpleLinearRegression:
    def __init__(self):
        self.slope = 0
        self.intercept = 0

    def fit(self, x, y):
        # Calculate slope and intercept using the least squares formula
        x_mean = np.mean(x)
        y_mean = np.mean(y)
        num = np.sum((x - x_mean) * (y - y_mean))
        deno = np.sum((x - x_mean) ** 2)
        self.slope = num / deno
        self.intercept = y_mean - self.slope * x_mean

    def predict(self, x):
        # Predict the target variable
        return self.slope * x + self.intercept

    def evaluate(self, x, y):
        # Evaluate model using Mean Squared Error (MSE)
        y_pred = self.predict(x)
        mse = np.mean((y - y_pred) ** 2)
        return mse

# Generate a synthetic dataset for demo
np.random.seed(42)
x = 2 * np.random.rand(100, 1).flatten()
y = 4 + 3 * x + np.random.randn(100)

# Split dataset into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

# Train and evaluate the model
model = SimpleLinearRegression()
model.fit(x_train, y_train)

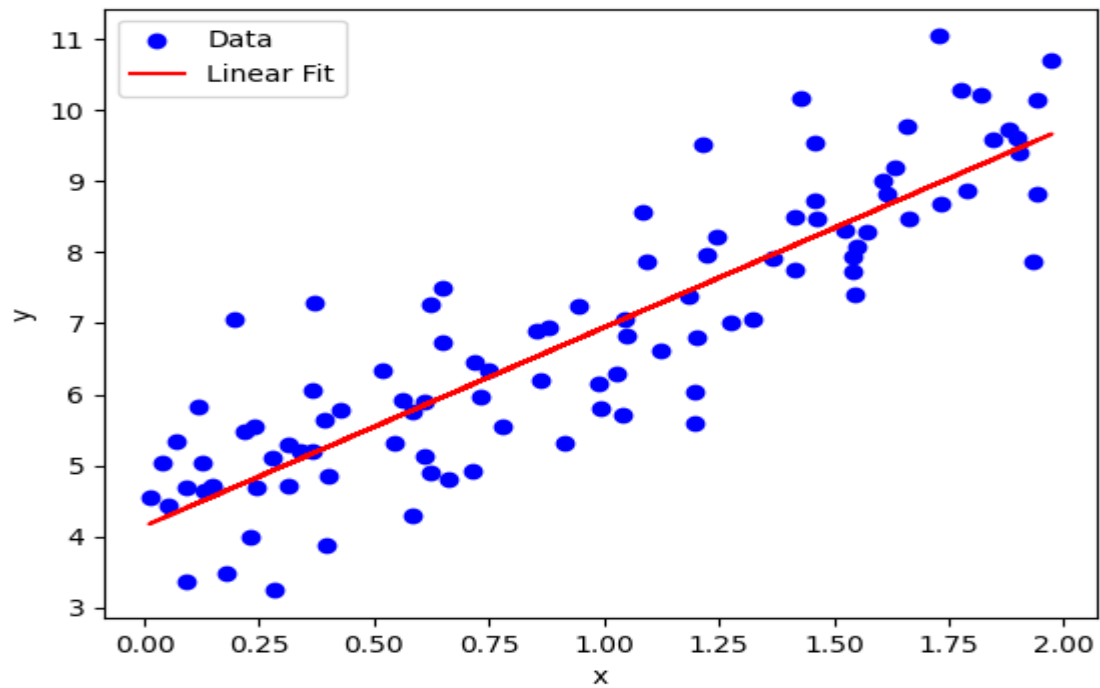
print(f'Slope: {model.slope}')
print(f'Intercept: {model.intercept}')

mse = model.evaluate(x_test, y_test)
print(f'Mean Squared Error on Test Set: {mse}')

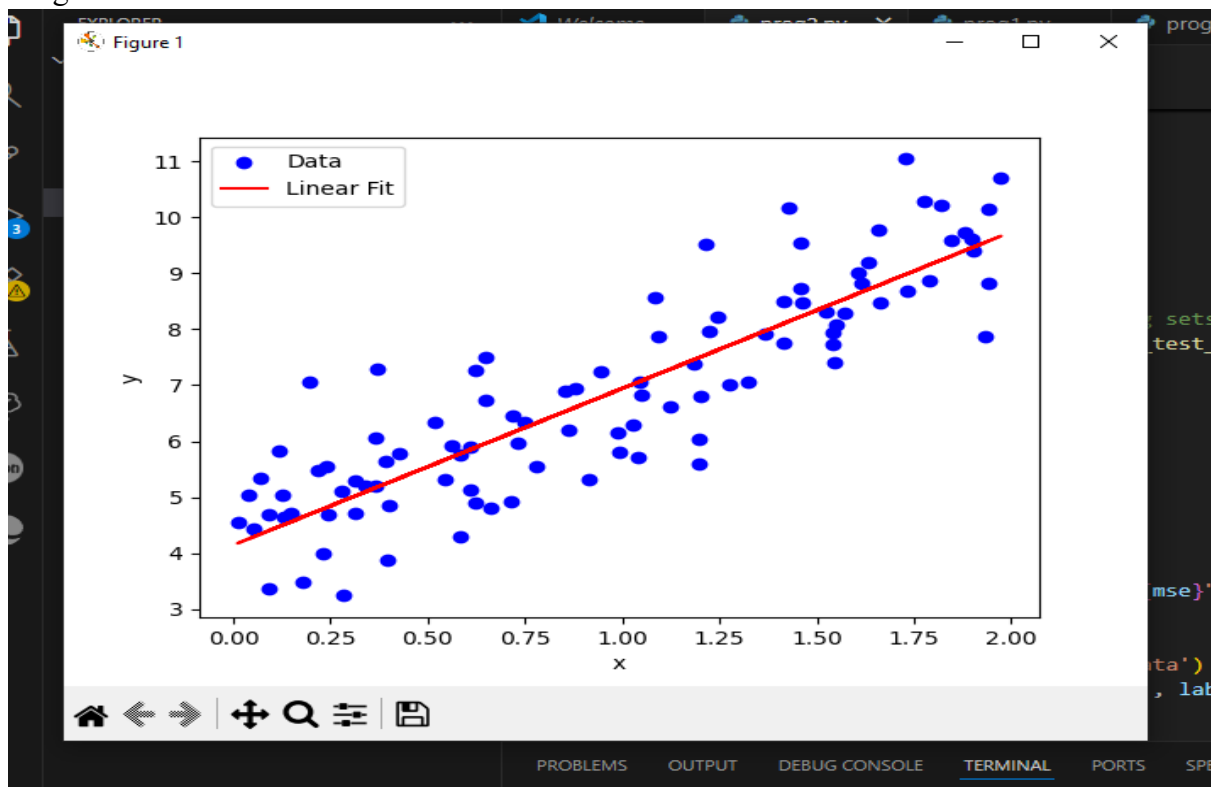
# Plot the results
plt.scatter(x, y, color='blue', label='Data')

```

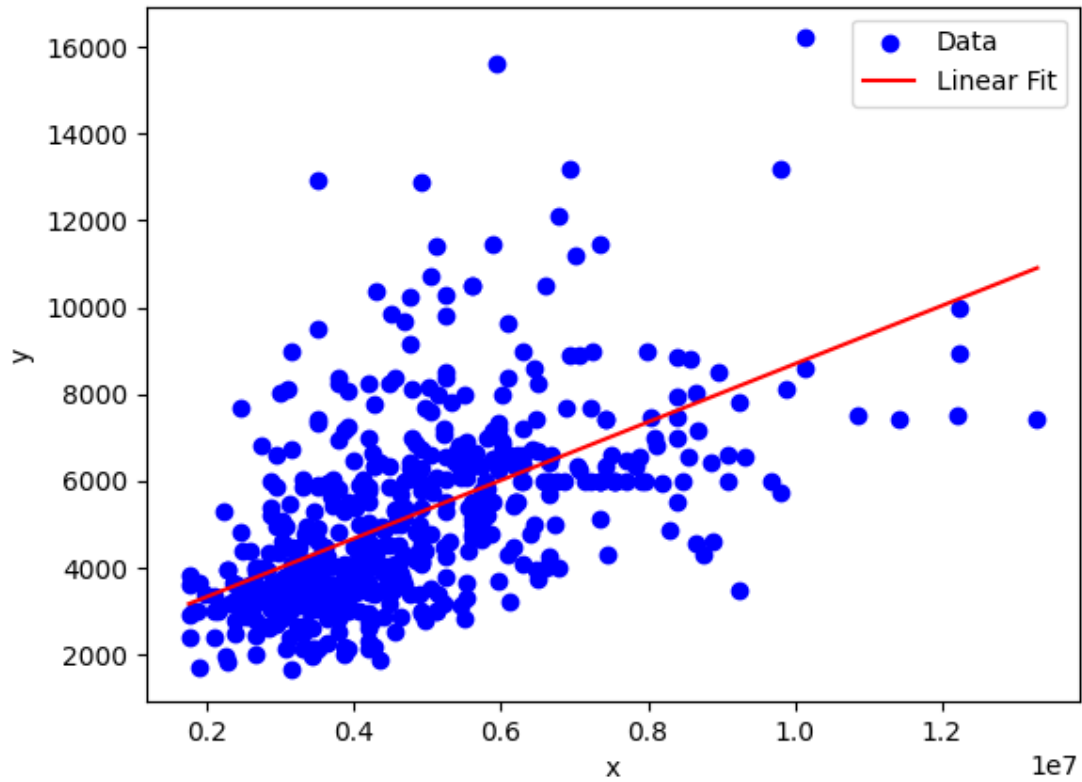
```
plt.plot(x, model.predict(x), color='red', label='Linear Fit')
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()
```



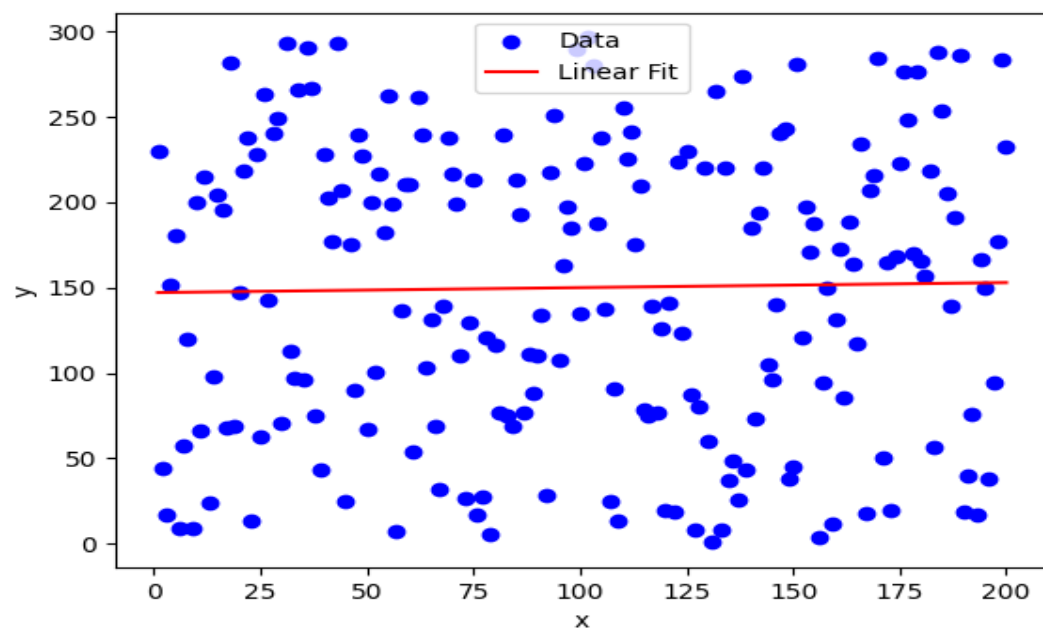
Prog2out



Houseprice



advertising



```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.svm import SVC

# Generate a synthetic dataset for binary classification
x, y = make_blobs(n_samples=100, centers=2, random_state=42, cluster_std=1.5)

# Train a support vector machine model
model = SVC(kernel='linear')
model.fit(x, y)

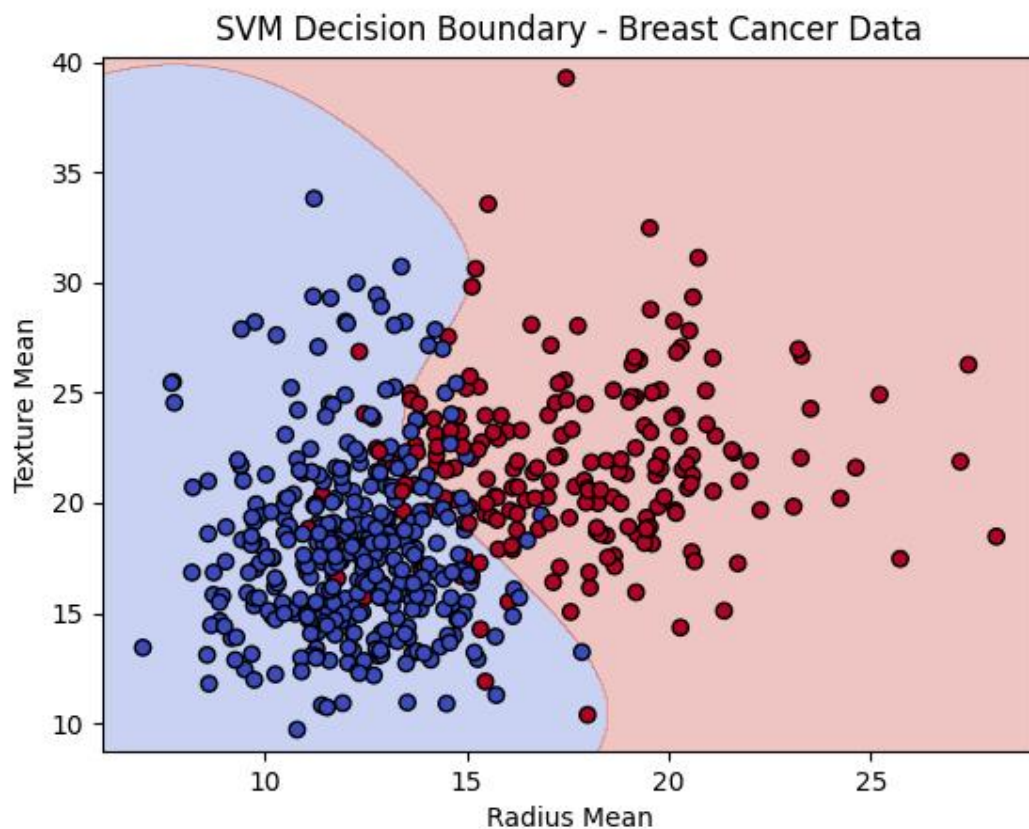
# Define a function to plot the decision boundary
def plot_decision_boundary(x, y, model):
    x_min, x_max = x[:, 0].min() - 1, x[:, 0].max() + 1
    y_min, y_max = x[:, 1].min() - 1, x[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01),
                          np.arange(y_min, y_max, 0.01))

    z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    z = z.reshape(xx.shape)

    plt.contourf(xx, yy, z, alpha=0.8, cmap=plt.cm.coolwarm)
    plt.scatter(x[:, 0], x[:, 1], c=y, edgecolors='k', cmap=plt.cm.coolwarm)
    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')
    plt.title('SVM Decision Boundary')
    plt.show() # Displays the plot

# Plot the decision boundary
plot_decision_boundary(x, y, model)
❖ ires
❖ breast cancer
❖ titanic
❖ pima indian diabetic dataset

```



```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# Load iris dataset
data = load_iris()
x = data.data      # feature matrix
y = data.target    # labels

# Split into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(x, y,
                                                    test_size=0.3,
                                                    random_state=42)

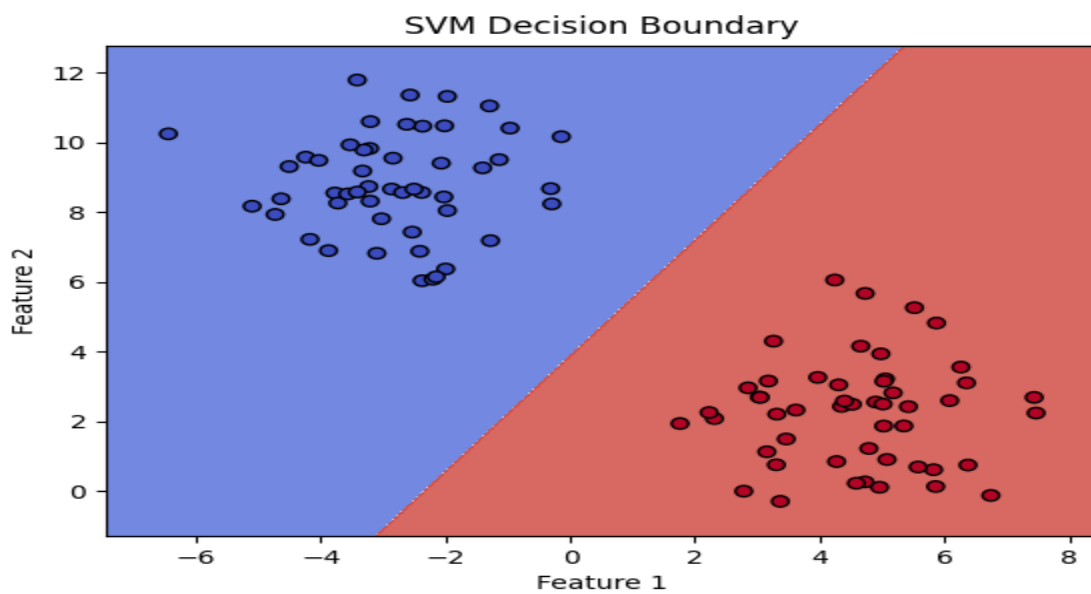
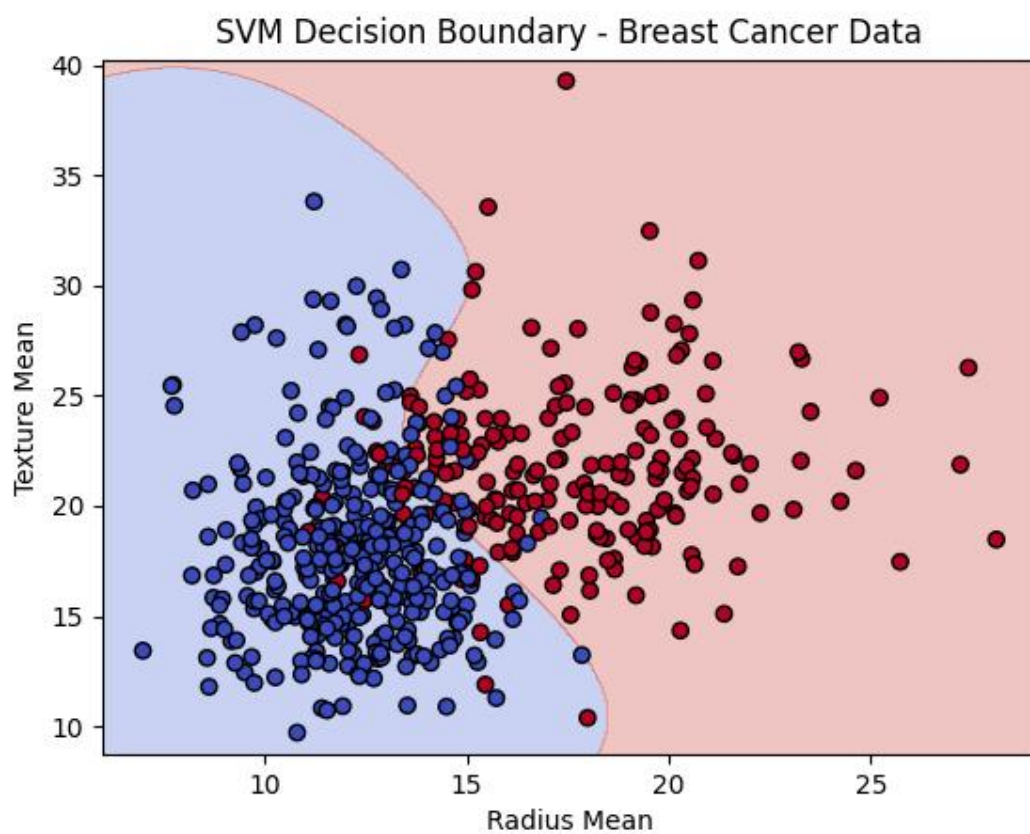
# Train Decision Tree using entropy (ID3)
id3_tree = DecisionTreeClassifier(criterion='entropy', random_state=42)
id3_tree.fit(x_train, y_train)

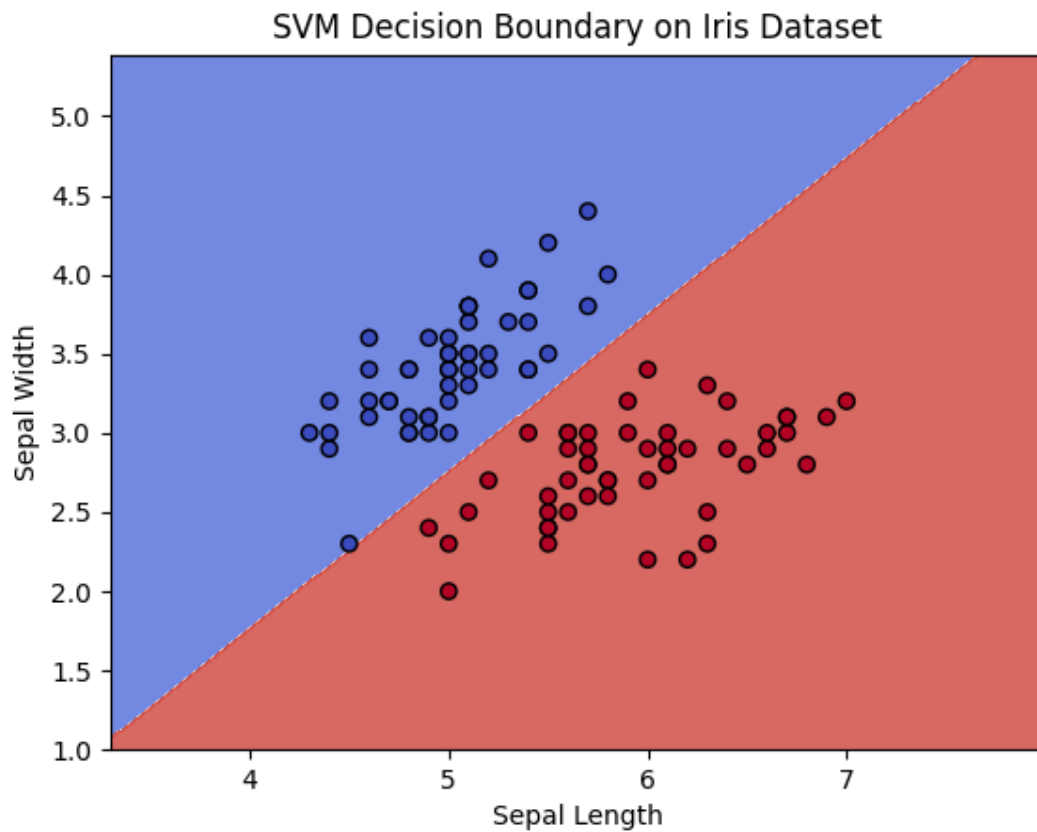
# Predictions
y_pred = id3_tree.predict(x_test)
```

```
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy of the ID3 Decision Tree: {accuracy*100:.2f} %')

# Display rules
print("\nDecision Tree Rules:")
print(export_text(id3_tree, feature_names=data.feature_names))

# Plot the tree
plt.figure(figsize=(12, 8))
plot_tree(id3_tree,
          feature_names=data.feature_names,
          class_names=data.target_names,
          filled=True)
plt.title("ID3 Decision Tree")
plt.show()
```



```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
```

```
# Dataset
```

```
data = np.array([
    [1200, 200],
    [1500, 250],
    [1700, 200],
    [2100, 400],
    [2300, 450],
    [2500, 500]
])
```

```
x = data[:, 0].reshape(-1, 1)
y = data[:, 1]
```

```
# Train-test split
```

```
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.3, random_state=42
)
```

```
# Custom KNN regression function
```

```
def knn_regression(x_train, y_train, x_test, k=3):
```

```

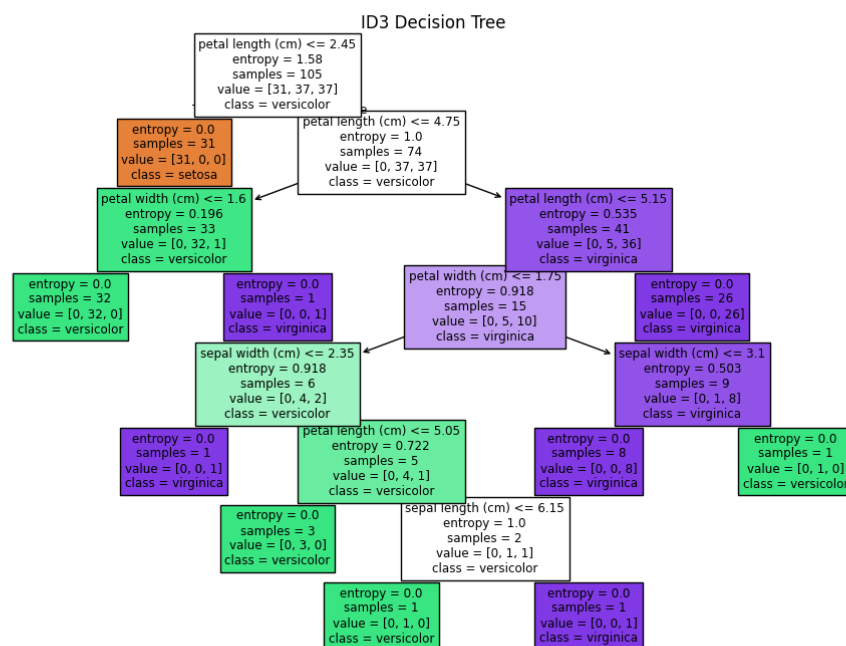
predictions = []
for test_point in x_test:
    distances = np.sqrt(np.sum((x_train - test_point)**2, axis=1))
    nearest_indices = np.argsort(distances)[:k]
    nearest_values = y_train[nearest_indices]
    prediction = np.mean(nearest_values)
    predictions.append(prediction)
return np.array(predictions)

# Predict
y_pred = knn_regression(x_train, y_train, x_test, k=3)

# MSE
mse = mean_squared_error(y_test, y_pred)
print(f"Mean squared error: {mse:.2f}")

# Predictions
print("\nTest Predictions:")
for size, actual, pred in zip(x_test.flatten(), y_test, y_pred):
    print(f"House Size: {size}, Actual price: {actual}, Predicted price: {pred:.2f}")

```



```

import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

```

```

import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
class SimpleLinearRegression:
    def __init__(self):
        self.slope=0
        self.intercept=0
    def fit(self,x,y):
        x_mean=np.mean(x) #calc slope and intercept using formula
        y_mean=np.mean(y)
        num=np.sum((x-x_mean)*(y-y_mean))
        deno=np.sum((x-x_mean)**2)
        self.slope=num/deno
        self.intercept=y_mean-self.slope*x_mean
    def predict(self,x):
        return self.slope*x+self.intercept #predict target variable
    def evaluate(self,x,y):
        y_pred=self.predict(x) #evaluate model using mean squared error
        mse=np.mean((y-y_pred)**2)
        return mse

#generate a synthetic dataset for demo
#np.random.seed(42)
#x=2*np.random.rand(100,1).flatten()
#y=4+3*x+np.random.randn(100)

#from sklearn.datasets import load_diabetes
#diabetes=load_diabetes()
#x=diabetes.data[:,2] #use one feature for simplicity
#y=diabetes.target
data = pd.read_csv('Housing.csv')
x = data.iloc[:, 0].values # Reshape for scikit-learn
y = data.iloc[:, 1].values

#split dataset into training and testing sets
x_train, x_test, y_train, y_test= train_test_split(x,y,test_size=0.2, random_state=42)
#train and evaluate the model
model=SimpleLinearRegression()
model.fit(x_train, y_train)

print(f'slope: {model.slope}')
print(f'Intercept: {model.intercept}')

mse =model.evaluate(x_test, y_test)
print(f'Mean Squared Error on Test Set: {mse}')

#plot the results
plt.scatter(x, y,color='blue', label='Data')

```

```

plt.plot(x, model.predict(x), color='red', label='Linear Fit')
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.preprocessing import LabelEncoder

# -----
# Load your Kaggle dataset
# -----
df = pd.read_csv("data.csv") # <-- replace with your filename

# -----
# Replace these with the EXACT column names in your dataset
# Example names:
# "radius_mean"
# "texture_mean"
# "diagnosis" (M/B)
# or "class" (0/1)
# -----
X = df[['radius_mean', 'texture_mean']].values
y_raw = df['diagnosis'] # <-- change if different column

# Encode labels automatically (works for M/B or 0/1)
le = LabelEncoder()
y = le.fit_transform(y_raw)

# -----
# Train Linear SVM
# -----
model = SVC(kernel='poly')
model.fit(X, y)

# -----
# Plot decision boundary
# -----
def plot_decision_boundary(X, y, model):
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

    xx, yy = np.meshgrid(
        np.arange(x_min, x_max, 0.05),

```

```

        np.arange(y_min, y_max, 0.05)
    )

    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)
    plt.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap=plt.cm.coolwarm)
    plt.xlabel("Radius Mean")
    plt.ylabel("Texture Mean")
    plt.title("SVM Decision Boundary - Breast Cancer Data")
    plt.show()

plot_decision_boundary(X, y, model)

import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix
import matplotlib.pyplot as plt

# Load dataset
data = pd.read_csv("Heart.csv")

# Drop unnecessary index column
if 'Unnamed: 0' in data.columns:
    data = data.drop('Unnamed: 0', axis=1)

# Features and target
X = data.drop("AHD", axis=1)
y = data["AHD"].map({'Yes': 1, 'No': 0})

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Decision Tree
id3_tree = DecisionTreeClassifier(criterion='entropy', random_state=42)
id3_tree.fit(X_train, y_train)

# Predictions
y_pred = id3_tree.predict(X_test)

# Accuracy
print(f'Accuracy: {accuracy_score(y_test, y_pred)*100:.2f}%')
```

```

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Export rules
print("\nDecision Tree Rules:")
print(export_text(id3_tree, feature_names=list(X.columns)))

# Plot tree
plt.figure(figsize=(20,10))
plot_tree(
    id3_tree,
    feature_names=list(X.columns),
    class_names=["No Disease", "Disease"],
    filled=True,
    rounded=True,
    fontsize=10
)
plt.show()


import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, accuracy_score

# -----
# 1. Load Dataset
# -----

data = pd.read_csv("Tshirt Dataset.csv") # change filename if needed

# Print column names for confirmation
print("Columns in dataset:", data.columns)
print(data.head())

# -----
# 2. Features and Labels (Correct columns)
# -----

X = data[['Height (in cms)', 'Weight (in kgs)']].values
y = data['T Shirt Size'].values

# -----
# 3. Train / Test Split

```

```

# -----

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# -----
# 4. KNN Classifier
# -----

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

# -----
# 5. Predictions
# -----

y_pred = knn.predict(X_test)

# -----
# 6. Evaluation
# -----

print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# -----
# 7. Print predictions
# -----

print("\nTest Predictions:")
for (h, w, actual, pred) in zip(X_test[:, 0], X_test[:, 1], y_test, y_pred):
    print(f"Height: {h}, Weight: {w} → Actual: {actual}, Predicted: {pred}")

```